01 / 20 — Phase 1 Assessment: React Fundamentals

Covers: lessons 01 through 19 of Phase 1. **Level:** assessment (no new material). **Time:** 3.5–4.5 hours (Parts A and B ≈ 1 h; Part C ≈ 1.5 h; Part D ≈ 1 h; Part E ≈ 45 min). **Total points:** 160. **Gate to Phase 2:** at least **80% overall (128/160)** and at least **70% in each of four categories (28/40 each)**.

---

1. How to take this assessment

1. **Environment.** Work in the Vite + React + TypeScript project from Lesson 19 with React DevTools installed (Lesson 18). Part C is implemented in code and must pass `npx tsc -b` and `npm run lint` with no errors.
2. **First pass is closed-book.** Do not consult the lessons, docs, or an AI assistant while answering. Looking things up is excellent learning practice, but it invalidates the *measurement*. You may use the editor, the compiler, the browser, and DevTools for Parts C and D (running code is not "looking up").
3. **Predict before you run.** In Part B, write your prediction down *before* running the code. Running it afterwards is a learning step, not a scoring tool: score your written prediction.
4. **Write explanations in your own words** (2–5 sentences for Part A). One-word answers receive no credit.
5. **Grade yourself strictly** using the answer key in §8 and the partial-credit rules in §10. If you are unsure whether you earned a point, you did not.
6. **Then** use §11 to turn weak areas into a study plan, and retake the relevant parts after a gap of at least 48 hours (the questions are the same, so also try to explain the *answers* to someone, which is the real test).

---

2. Scoring structure

Category	Parts	Points	Pass threshold (70%)
Explanation	Part A (20 questions × 2 points)	40	28
Reasoning	Part B (10 predictions × 2) + Part E (4 questions × 5)	40	28
Implementation	Part C (4 tasks × 10)	40	28
Debugging	Part D (5 challenges × 8)	40	28
Total		**160**	**128 (80%)**

---

3. Part A — Conceptual questions (Explanation, 40 points)

Answer each in 2–5 sentences. Each is worth 2 points.

A1 *(Lesson 01).* React is often called "a library, not a framework". State what React itself provides and name three things a typical app needs that React does not provide.

A2 *(02).* Explain the difference between imperative and declarative UI code, using "show an error message under an input" as the example. What responsibility does the declarative style remove from the programmer?

A3 *(03).* What does "UI is a function of state" mean? Give one bug that appears when you *also* modify the DOM manually (outside React) in a React-managed subtree.

A4 *(04).* Distinguish a React **element**, a **component**, and a component **instance**. Which of the three holds state?

A5 *(05).* What does JSX compile to? Why must component names start with a capital letter, and why is the HTML attribute `class` written `className` in JSX?

A6 *(06).* What is one-way data flow? Why are props read-only, and what do you do when a child needs to cause a change in the parent's data?

A7 *(07).* What is the `children` prop, and how does composition (passing JSX as children or as props) reduce "prop drilling" without any state-management library?

A8 *(08).* Why can `{count && <Badge />}` display a literal `0`? Give two correct alternatives.

A9 *(09).* What are keys for? Why is array index a risky key for dynamic lists, and what makes a good key?

A10 *(10).* In React's event system, what is wrong with `onClick={handle(id)}`, and what are two correct forms? Roughly where does React attach DOM listeners, and what does that imply about `e.stopPropagation()` interacting with non-React listeners? *(Answer at the level taught in Lesson 10.)*

A11 *(11).* Compare controlled and uncontrolled inputs: where does the "source of truth" for the value live, and when would you choose each?

A12 *(12).* What does it mean for a component to be pure? List two things that must **not** happen during render, and where side effects belong instead.

A13 *(13).* Describe the three steps of React's rendering mental model (trigger, render, commit). Why is "the component rendered" not the same as "the DOM changed"?

A14 *(14).* Explain "state as a snapshot". If a handler calls `setCount(count + 1)` and then `alert(count)`, what is alerted and why?

A15 *(15).* What is an updater function, and when must you use one? What is batching, and what changed about it in React 18?

A16 *(16).* State the rule that decides whether React preserves or resets a component's state. Give two ways to force a reset.

A17 *(17).* Describe mount, update, and unmount. What does an effect's cleanup function do and when does it run? Why does Strict Mode run effects "setup → cleanup → setup" in development?

A18 *(18).* In React DevTools, what is the difference between a component's parent and its owner? Why would a flash under "highlight updates" not prove the DOM changed?

A19 *(19).* Why does `npm run dev` not report TypeScript errors, and where are they reported? What does the `VITE_` prefix do, and why is it unsuitable for secrets?

A20 *(cross-cutting: 06, 12, 13).* A parent re-renders and passes the same props to a child. Does the child render? Why? Name the (later-lesson) tool people reach for, and one reason not to reach for it first.

---

4. Part B — Code-prediction tasks (Reasoning, 20 points)

Write your prediction **before** running. Each is worth 2 points: 1 for the correct result, 1 for the correct reason. Assume React 19 with `createRoot` and **no** `<StrictMode>` unless stated.

B1. After one click, what does the button display, and what does the console log?

```
function App() {
  const [n, setN] = useState(0);
  return (
    <button
      onClick={() => {
        setN(n + 1);
        setN(n + 1);
        setN(p => p + 1);
        console.log(n);
      }}
    >
      {n}
    </button>
  );
}
```

B2. Starting at `n = 0`, one click runs `setN(p => p + 2); setN(3); setN(p => p * 10);`. What does `n` become?

B3. What appears on screen? (`Results` renders a list.)

```
function Page({ items }: { items: string[] }) {
  return <div>{items.length && <Results items={items} />}</div>;
}
// <Page items={[]} />
```

B4. Uncontrolled inputs inside keyed rows. The user types `x` into the first row's input, then clicks "Remove first". What do the two remaining rows show (label and input content)?

```tsx
function Row({ label }: { label: string }) {
  return <li>{label}: <input aria-label={label} /></li>;
}
function App() {
  const [items, setItems] = useState(["A", "B", "C"]);
  return (
    <>
      <ul>{items.map((l, i) => <Row key={i} label={l} />)}</ul>
      <button onClick={() => setItems(p => p.slice(1))}>Remove first</button>
    </>
  );
}
```

B5. A `Counter` holds `useState(0)`. The user clicks it to 3, then `flag` toggles. What does the counter show after toggling for each variant?

```tsx
// (i)   {flag ? <Counter /> : <Counter />}
// (ii)  {flag ? <Counter key="a" /> : <Counter key="b" />}
// (iii) {flag ? <div><Counter /></div> : <section><Counter /></section>}
```

B6. The log output (production) for: mount with `id = 1`, then the parent sets `id = 2`, then the parent unmounts `Probe`.

```tsx
function Probe({ id }: { id: number }) {
  console.log("render", id);
  useEffect(() => {
    console.log("setup", id);
    return () => console.log("cleanup", id);
  }, [id]);
  return null;
}
```

B7. In **development with** `<StrictMode>`, what do the effect logs show on initial mount only?

```tsx
useEffect(() => { console.log("on"); return () => console.log("off"); }, []);
```

B8. Describe what the user sees after one click, and after a later unrelated re-render (for example, a different piece of state changes).

```tsx
const [items, setItems] = useState<string[]>(["a"]);
// <button onClick={() => { items.push("b"); setItems(items); }}>Add</button>
// <ul>{items.map(i => <li key={i}>{i}</li>)}</ul>
```

B9. The user focuses the field and types `x`. What does the field contain? Does React report anything?

```
<input value="abc" />
```

B10. The component logs `"render"` in its body. After the initial render, how many times is `"render"` logged following a single click? Why?

```
const [a, setA] = useState(0);
const [b, setB] = useState(0);
console.log("render");
// onClick: () => { setA(1); setB(1); setTimeout(() => { setA(2); setB(2); }, 0); }
```

---

5. Part C — Implementation exercises (Implementation, 40 points)

Each task is worth 10 points. Implement in your Vite project in TypeScript with strict mode. Each must compile (`npx tsc -b`) and pass lint. Rubrics are in the answer key; read the requirements carefully.

C1. `ProductList` **(typed props, lists, conditional rendering, events, accessibility).**

```
type Product = { id: string; name: string; priceCents: number; inStock: boolean };
```

Build `ProductList({ products, onAddToCart })` : a list of products showing name and price formatted as currency; an "Add to cart" button per product that calls `onAddToCart(id)` ; the button is disabled and reads "Out of stock" when `inStock` is false; an empty-state message when there are no products. Use proper keys, no state, no mutation, accessible button names.

C2. `SignupForm` **(controlled inputs, derived values, form events).** A controlled form with email and password. Validation: email must look like an address; password at least 8 characters. Errors appear **only after the first submit attempt** and update as the user types afterwards. On a valid submit, call a typed `onSubmit({ email, password })` prop; invalid submits must not call it. Do not store errors in state (derive them). Use a `<form>` with an `onSubmit` handler (typed event, `preventDefault`), `label` / `htmlFor` associations, and announce errors accessibly.

C3. `TaskList` **(state updates, immutability, updaters).** Add a task via a form (trimmed, non-empty); toggle done; remove; show "N remaining". Use immutable updates and updater functions where the new state depends on the old. No `useEffect` . The remaining count must be derived, not stored.

C4. `Stopwatch` **(effects, cleanup, updaters).** Start/Stop and Reset buttons; displays elapsed seconds with one decimal (tenths). It must not leak intervals (including under Strict Mode and when unmounted), must not read stale state inside the interval, and must keep its elapsed time when stopped and restarted.

---

6. Part D — Debugging challenges (Debugging, 40 points)

Each is worth 8 points: **2** for identifying the symptom's cause, **2** for explaining the mechanism, **2** for a correct fix, **2** for stating how you would verify the fix (DevTools/logs/tests). There may be more than one defect where indicated.

D1. "The input loses focus after every keystroke."

```
export function Search() {
  const [q, setQ] = useState("");
  const Box = () => <input value={q} onChange={e => setQ(e.target.value)} aria-label="Search"
/>;
  return <div><Box /><p>Searching for: {q}</p></div>;
}
```

D2. "The timer shows 1 and stops, and it keeps running after I leave the page."

```
export function Elapsed() {
  const [seconds, setSeconds] = useState(0);
  useEffect(() => {
    setInterval(() => setSeconds(seconds + 1), 1000);
  }, []);
  return <p>{seconds}s</p>;
}
```

Describe the behaviour in development with StrictMode too.

D3. "Deleting a row removes the wrong note, and sometimes nothing changes." (Two defects.)

```
function NoteRow({ title }: { title: string }) {
  const [note, setNote] = useState("");
  return <li>{title} <input value={note} onChange={e => setNote(e.target.value)} aria-label=
{`Note for ${title}`} /></li>;
}
export function Notes({ initial }: { initial: string[] }) {
  const [titles, setTitles] = useState(initial);
  function remove(i: number) {
    titles.splice(i, 1);
    setTitles(titles);
  }
  return (
    <ul>
      {titles.map((t, i) => (
        <div key={i}>
          <NoteRow title={t} />
          <button onClick={() => remove(i)}>Delete {t}</button>
        </div>
      ))}
    </ul>
  );
}
```

D4. "After selecting another user, the form shows the previous user's name."

```
function UserForm({ user }: { user: { id: string; name: string } }) {
  const [name, setName] = useState(user.name);
  return <input value={name} onChange={e => setName(e.target.value)} aria-label="Name" />;
}
export function Page({ users }: { users: { id: string; name: string }[] }) {
  const [id, setId] = useState(users[0].id);
  const user = users.find(u => u.id === id)!;
  return (
    <>
      <select value={id} onChange={e => setId(e.target.value)} aria-label="User">
        {users.map(u => <option key={u.id} value={u.id}>{u.name}</option>)}
      </select>
      <UserForm user={user} />
    </>
  );
}
```

D5. "The page crashes with 'Too many re-renders'."

```
export function Like() {
  const [likes, setLikes] = useState(0);
  return <button onClick={setLikes(likes + 1)}>Likes: {likes}</button>;
}
```

Also state: what would `onClick={() => setLikes(likes + 1)}` and `onClick={() => setLikes(l => l + 1)}` each do on a double-click that registers two click events before React renders? (Explain whether the two differ.)

---

7. Part E — Architecture and reasoning questions (Reasoning, 20 points)

Each is worth 5 points. Answer in 6–12 sentences; justify with criteria and mention at least one trade-off and one case where your recommendation would be wrong.

E1 (state placement). A page has a search box, a results list that depends on the search text, and a large chart that does *not* depend on the search text. Typing feels sluggish because the chart re-renders on each keystroke. Explain the cause and propose two different structural fixes; recommend one.

E2 (controlled vs uncontrolled). You must build (a) a 3-field newsletter signup and (b) a 40-field admin configuration form with conditional sections and live validation. For each, pick controlled, uncontrolled, or a mix, and justify.

E3 (preserve vs reset). A messaging UI shows one conversation at a time, with a draft message box. Requirements differ across teams: (1) drafts must reset when switching conversations; (2) drafts must be

remembered per conversation; (3) drafts must survive a page reload. Propose structure for each, naming the React mechanism (key, lifted state, storage) and the main risk of each.

E4 (code review). A teammate's PR does the following: stores `filteredItems` in state and updates it with a `useEffect` whenever `items` or `query` changes; and triggers an analytics call from an effect that watches `submitted` state. Write the review feedback: what is wrong, why, and what you would change. Include the one scenario in which an effect *is* the right tool nearby.

---

8. Answer key

> Grade yourself after completing every part. Keep the scoring rules in §10 in mind.

8.1 Part A answer key (2 points each)

A1. React provides a model and runtime for describing UI as components, state, and props, plus `react-dom` for the DOM host: it renders and updates UI in response to state. It does not provide routing, data fetching/caching, a build toolchain, or application structure (also: form libraries, global state management, styling system, server rendering orchestration without a framework). *Full credit:* the core rendering/state model plus three non-provided concerns. *Partial (1):* names three non-provided things but does not say what React does provide, or the reverse. *Zero:* "React is a framework that does everything".

A2. Imperative: you write the *steps* to change the existing DOM (create a `<p>`, set text, insert after input; later find and remove it). Declarative: you describe what the UI should look like for the current state (`{error && <p role="alert">{error}</p>}`), and React works out the DOM changes. It removes the need to track the current DOM and write every transition (add/remove/update ordering) by hand, preventing drift between state and screen. *Partial:* correct definitions without the responsibility removed.

A3. The UI you see is computed from state by your render functions; to change the UI, change state. If you also edit the DOM manually inside a React-managed subtree (e.g., `element.textContent = "x"` or removing a node), React's idea of the DOM and the real DOM diverge: the next render may overwrite your change, ignore it, or throw errors when it tries to update/remove nodes that no longer exist or that you changed. Any concrete divergence bug earns the second point.

A4. An **element** is a plain object describing what to render (`type`, `props`, `key`), created by JSX/`createElement`; it's immutable and cheap. A **component** is a function (or class) that takes props and returns elements. An **instance** is React's internal record for a component rendered at a particular position in the tree; **instances hold state** (and hooks). (Function components have no `this`-instances in the class sense, but the position-bound state record is the instance concept.)

A5. JSX compiles to calls to the JSX runtime / `createElement` producing element objects: `<Foo a={1}/>` → `jsx(Foo, { a: 1 })`. Lowercase names are treated as host (DOM) tags (strings); capitalised names refer to component values (variables in scope). `class` is a reserved word in JavaScript (and JSX props map to DOM properties, in which `className` is the property name), so JSX uses `className`. *Full credit requires:* compile target, capitalisation rule reason, `className` reason.

A6. Data flows downward: parents pass props to children; children cannot modify props. Props are read-only so components are pure functions of their inputs and data has a single owner; if children could mutate, changes would be untraceable and renders unpredictable. For a child to cause a change, the parent passes a callback prop (e.g., `onSelect`), the child calls it, and the parent updates its own state ("lifting state up"/events up).

A7. `children` is the prop containing whatever JSX is nested between a component's tags. Composition lets a parent create the elements and hand them to a wrapper (`<Layout><Sidebar user={user} /></Layout>`) so intermediate components needn't forward props they don't use, shortening prop chains. Passing JSX via named props ("slots") extends this. It works because the owner creates the elements with the data in hand. *Partial:* defines `children` but doesn't connect it to prop drilling.

A8. `0 && <Badge />` evaluates to `0`, and React renders numbers (only `false`, `null`, `undefined`, `true` render nothing). Alternatives: `{count > 0 && <Badge />}`; `{count ? <Badge /> : null}`; early return; `Boolean(count) && ...`. Any two correct ones.

A9. Keys give elements stable identity among siblings so React can match items between renders (preserve state, DOM, and reorder correctly) instead of matching by position. Index keys break when items are inserted, removed, or reordered: state and DOM stay with the *position*, producing data mixed between rows. A good key is unique among siblings, stable across renders, and derived from the data (an id), not generated per render (`Math.random()`).

A10. `handle(id)` is *called during render* and its return value is passed as the handler (so it runs at the wrong time and may loop if it sets state). Correct: `onClick={() => handle(id)}` or a handler that returns a function (`onClick={makeHandler(id)}`), or `onClick={handle}` if no args needed. React (17+) attaches its listeners at the **root container** and uses event delegation with synthetic events; Implication: native listeners attached to elements *inside* the root container fire before React's handlers (the event must bubble up to the root first), while `e.stopPropagation()` in a React handler stops further propagation from the root upward, so listeners on `document` or `window` do not receive that event. (Grading: full credit for the wrong-call explanation, two valid forms, root delegation, and the implication. If you answered only the first two parts, 1 point.)

A11. Controlled: React state is the source of truth; the input's `value` is set from state and `onChange` updates it; allows validation, formatting, and conditional UI on each keystroke. Uncontrolled: the DOM holds the value; you read it on demand (ref/`FormData`) with `defaultValue` for initial; less code and fewer renders for simple forms, file inputs are always uncontrolled. Choose controlled when the UI depends on the value as the user types; uncontrolled for simple submit-time reads or very large forms (or use a form library).

A12. A pure component returns the same output for the same props/state/context and has no observable side effects during render. Must not: mutate props/state/variables created outside render, perform I/O (fetch, writing to the DOM/storage), start timers/subscriptions, or use non-deterministic values (`Math.random()`, `Date.now()`) in output that should be stable. Side effects belong in event handlers (for user actions) or in effects (for synchronization with external systems).

A13. *Trigger* (initial render or a state update), *render* (React calls components to compute the new element tree; must be pure), *commit* (React applies the minimal DOM changes and runs layout effects then

effects). A render may produce an identical result, in which case the commit doesn't touch the DOM. So "rendered" means the function ran and React computed output; the DOM changes only for differences.

A14. Each render gets its own fixed props, state, and handlers (a snapshot). `setCount(count + 1)` requests an update for the *next* render; the running handler's `count` is a constant. `alert(count)` shows the **old** value (e.g., 0) because that closure captured the snapshot; the new value appears in the next render. (Even in `setTimeout`, the closure keeps the old value.)

A15. An updater function is `setX(prev => next)`: React calls it with the latest queued state, so it composes correctly when several updates are pending or when the next state depends on the previous. Use it for counters, toggles, list edits. Batching is processing multiple updates together in one render. React 18+ batches automatically in all contexts (timeouts, promises, native handlers) with `createRoot`; React 17 batched only inside React event handlers.

A16. State is preserved when the same component type (and key) is rendered at the same position in the tree; it's reset when the type or key at that position changes (or the component is removed). Two ways to reset: change the `key` (`<Form key={id} />`), or render a different component type/wrapper at that position (or unmount and remount via conditional). (Resetting by effect is possible but discouraged.)

A17. Mount: first creation and commit of the instance; update: re-render of an existing instance; unmount: removal and state discard. An effect's cleanup undoes the setup (disconnect, clear timer, remove listener) and runs before the effect re-runs due to changed dependencies and when the component unmounts. Strict Mode runs setup → cleanup → setup on mount in development to simulate remounting and reveal effects that don't clean up correctly; production doesn't.

A18. The *parent* is the tree container; the *owner* is the component whose render created the element (important for `children` passed through wrappers; "where did these props come from?" is answered by the owner). A flash means the component *rendered* (function was called/commit involved it), which doesn't imply any DOM mutation: React may compare and find no differences.

A19. Vite only transpiles and strips types (per file, for speed); it doesn't type-check. Errors are reported by the editor's TypeScript service, `tsc` (`npx tsc -b`), and `npm run build` (which runs `tsc -b` first), plus CI. Variables prefixed `VITE_` are exposed to client code via `import.meta.env` and inlined into the bundle at build time, so every user can read them; secrets must stay on a server.

A20. Yes, the child renders by default: when a parent renders, React renders its children (calls their functions) regardless of whether props changed, because React doesn't compare props unless told to. Tool: `React.memo` (skips re-rendering when props are shallow-equal). Reason not to reach for it first: rendering is often cheap; memo adds comparison cost and complexity, fails if props are new object/function identities each render, and optimisation should follow measurement (Profiler) and structural fixes (colocating state).

8.2 Part B answer key (2 points each: 1 result + 1 reason)

B1. Displays **2**; logs **0**. Snapshot `n = 0`: the first two calls enqueue "replace with 1"; the updater makes it 2; `console.log(n)` prints the snapshot (0).

B2. `0 → 2 → 3 → 30`; **30**. The replacement `3` overrides the earlier result; the final updater multiplies the running value.

B3. The page shows `0`. `[].length` is `0`; `0 && X` evaluates to `0`, which React renders as text. Fix: `items.length > 0 && ...`.

B4. Rows show: `B:` with input containing `x`, and `C:` with an empty input. Keys are indices `0, 1, 2`. After removal, keys are `0, 1`; the instances/DOM for keys 0 and 1 persist (the uncontrolled input DOM keeps its value) and receive new labels `B` and `C`; the instance for key 2 is unmounted. The typed `x` stays at position 0, now labelled B. (Stable id keys would remove A's input along with its text.)

B5. (i) **3** (same type, position, key: preserved). (ii) **0** (different keys: remount). (iii) **0** (`div` → `section` changes the type at that position, so the subtree is replaced). All three correct = 2 points; 2 of 3 with right reasoning = 1.

B6. `render 1`, `setup 1`, `render 2`, `cleanup 1`, `setup 2`, `cleanup 2`. The cleanup after the prop change uses the previous render's closure (id 1); the final cleanup on unmount uses id 2.

B7. `on`, `off`, `on`. Strict Mode (development) simulates unmount/remount after the first mount to check cleanup symmetry; production logs only `on`.

B8. Nothing visible changes after the click: `push` mutated the existing array and `setItems(items)` passes the *same reference*, so React's `Object.is` check bails out (no re-render). After a *later unrelated re-render* (another state change), the component renders with the mutated array and shows `a` and `b`. Fix: `setItems(p => [...p, "b"])`.

B9. The field **stays** `abc`; typing does nothing visible. React warns in development that a `value` prop was provided without an `onChange` handler (the field is read-only) and suggests `defaultValue` (uncontrolled) or `onChange` (controlled) or `readOnly`. (Credit if you noted the warning *or* the controlled-without-handler reasoning clearly.)

B10. 2 logs after the click. The first batch (`setA(1); setB(1)` in the event handler) yields one render. The `setTimeout` callback runs in a later task; with automatic batching (React 18+) its two updates are one batch, yielding one more render. (React 17 behaviour in the timeout would be 2 renders, 3 total; this question assumes React 19.)

8.3 Part C answer key and rubrics (10 points each)

C1 reference.

```tsx
type Product = { id: string; name: string; priceCents: number; inStock: boolean };

type ProductListProps = {
  products: readonly Product[];
  onAddToCart: (id: string) => void;
};

const currency = new Intl.NumberFormat("en-US", { style: "currency", currency: "USD" });

export function ProductList({ products, onAddToCart }: ProductListProps) {
  if (products.length === 0) {
    return <p>No products found.</p>;
  }
  return (
    <ul>
      {products.map(p => (
        <li key={p.id}>
          <span>{p.name}</span>{" "}
          <span>{currency.format(p.priceCents / 100)}</span>{" "}
          <button
            type="button"
            disabled={!p.inStock}
            onClick={() => onAddToCart(p.id)}
            aria-label={p.inStock ? `Add ${p.name} to cart` : `${p.name} is out of stock`}
          >
            {p.inStock ? "Add to cart" : "Out of stock"}
          </button>
        </li>
      ))}
    </ul>
  );
}
```

Rubric (10): typed props and `Product` (2); stable `key={p.id}` (2); empty state (1); disabled + text for out of stock (1); handler is a closure, not invoked in render (1); no state or mutation, formatting via `Intl.NumberFormat` or equivalent with cents→currency (1); accessible names / `type="button"` (1); compiles and lints (1). *Common errors:* index keys (–2), `onClick={onAddToCart(p.id)}` (–3), floating-point money (store cents) (–1), `{products.length && ...}` (–1). *Edge notes:* `readonly Product[]` accepts mutable arrays; the formatter is hoisted so it isn't recreated every render (an optimisation, not required).

C2 reference.

```tsx
import { useState, type FormEvent } from "react";

type Errors = { email?: string; password?: string };
type Values = { email: string; password: string };

function validate({ email, password }: Values): Errors {
  const errors: Errors = {};
  if (!/^\S+@\S+\.\S+$/.test(email)) errors.email = "Enter a valid email address.";
  if (password.length < 8) errors.password = "Password must be at least 8 characters.";
  return errors;
}

export function SignupForm({ onSubmit }: { onSubmit: (values: Values) => void }) {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [attempted, setAttempted] = useState(false);

  const errors = validate({ email, password });      // derived on every render
  const isValid = Object.keys(errors).length === 0;

  function handleSubmit(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    setAttempted(true);
    if (isValid) onSubmit({ email, password });
  }

  return (
    <form onSubmit={handleSubmit} noValidate>
      <div>
        <label htmlFor="signup-email">Email</label>
        <input
          id="signup-email"
          type="email"
          value={email}
          onChange={e => setEmail(e.target.value)}
          aria-invalid={attempted && errors.email !== undefined}
          aria-describedby={attempted && errors.email ? "signup-email-error" : undefined}
        />
        {attempted && errors.email && <p id="signup-email-error" role="alert">{errors.email}
</p>}
      </div>
      <div>
        <label htmlFor="signup-password">Password</label>
        <input
          id="signup-password"
          type="password"
```

```
          value={password}
          onChange={e => setPassword(e.target.value)}
          aria-invalid={attempted && errors.password !== undefined}
          aria-describedby={attempted && errors.password ? "signup-password-error" :
undefined}
        />
        {attempted && errors.password && <p id="signup-password-error" role="alert">
{errors.password}</p>}
      </div>
      <button type="submit">Create account</button>
    </form>
  );
}
```

Rubric (10): controlled inputs with `value` + `onChange` (2); errors **derived**, not stored (2); `attempted` flag gating display and persisting thereafter (1); `preventDefault` and typed `FormEvent` (1); invalid submit doesn't call `onSubmit` (1); labels linked with `htmlFor` / `id` (1); error association/announcement (`aria-describedby` , `role="alert"` or equivalent) (1); compiles and lints (1). *Common errors:* storing `errors` in state via `useEffect` (–2, derived-state smell); validating only in the `onChange` and leaving stale errors; `setAttempted(true)` then reading `attempted` in the same handler; using `any` for events. *Notes:* `noValidate` disables the browser's built-in validation UI so your messages are shown; real products must also validate on the server, and the email regex is a UX nicety, not verification.

C3 reference.

```tsx
import { useState, type FormEvent } from "react";

type Task = { id: string; title: string; done: boolean };

export function TaskList() {
  const [tasks, setTasks] = useState<Task[]>([]);
  const [title, setTitle] = useState("");

  const remaining = tasks.filter(t => !t.done).length;      // derived

  function handleAdd(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    const trimmed = title.trim();
    if (trimmed === "") return;
    const task: Task = { id: crypto.randomUUID(), title: trimmed, done: false }; // created in the handler (impure call stays out of the updater)
    setTasks(prev => [...prev, task]);
    setTitle("");
  }

  const toggle = (id: string) =>
    setTasks(prev => prev.map(t => (t.id === id ? { ...t, done: !t.done } : t)));

  const remove = (id: string) =>
    setTasks(prev => prev.filter(t => t.id !== id));

  return (
    <section>
      <form onSubmit={handleAdd}>
        <label>
          New task <input value={title} onChange={e => setTitle(e.target.value)} />
        </label>
        <button type="submit">Add</button>
      </form>
      <p aria-live="polite">{remaining} remaining</p>
      <ul>
        {tasks.map(t => (
          <li key={t.id}>
            <label>
              <input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} />
              {t.title}
            </label>
            <button type="button" onClick={() => remove(t.id)} aria-label={`Remove ${t.title}`}>×</button>
          </li>
        ))}
```

```
      </ul>
    </section>
  );
}
```

Rubric (10): immutable updates for add/toggle/remove (3); updater functions for the three list updates (2); derived `remaining` (1); ids created outside updaters and used as keys (1); trimmed/non-empty validation and input reset (1); accessible labelling (1); compiles and lints (1). *Common errors:* `tasks.push`, `tasks[i].done = ...` (–3); index keys (–1); `remaining` in state (–1); `crypto.randomUUID()` inside an updater (–1; impure under Strict Mode: two invocations produce different ids, only one is used, which is not harmful but is a smell, and for non-idempotent effects it is harmful). Note `crypto.randomUUID` needs a secure context (HTTPS or localhost).

C4 reference.

```
import { useEffect, useState } from "react";

export function Stopwatch() {
  const [running, setRunning] = useState(false);
  const [elapsedMs, setElapsedMs] = useState(0);

  useEffect(() => {
    if (!running) return undefined;
    const id = window.setInterval(() => setElapsedMs(ms => ms + 100), 100);
    return () => window.clearInterval(id);
  }, [running]);

  return (
    <section aria-label="Stopwatch">
      <p aria-live="off">{(elapsedMs / 1000).toFixed(1)} s</p>
      <button type="button" onClick={() => setRunning(r => !r)}>{running ? "Stop" : "Start"}
</button>
      <button type="button" onClick={() => { setRunning(false); setElapsedMs(0);
}}>Reset</button>
    </section>
  );
}
```

Rubric (10): effect depends on `[running]` and starts the interval only while running (2); cleanup clears the interval (2); updater `ms => ms + 100` (no stale read) (2); elapsed retained across stop/start (state outside the effect) (1); Reset stops and zeroes (1); formatting to tenths (1); compiles/lints, no `any` (1). *Strict Mode check:* in development, setup → cleanup → setup leaves exactly one interval. *Common errors:* `setInterval` without cleanup (–3); `setElapsedMs(elapsedMs + 100)` with `[running]` deps (stale closure, –2); `useEffect(async …)`. *Extension (not required):* ticks of 100 ms drift under load; a more accurate stopwatch stores a start timestamp and computes `performance.now() - start` on each tick, using the interval only to trigger renders.

8.4 Part D answer key (8 points each: cause 2, mechanism 2, fix 2, verification 2)

D1. *Cause:* `Box` is defined inside `Search`. *Mechanism:* each render of `Search` creates a new function, so React sees a different component type at that position and unmounts/mounts it, replacing the `<input>` DOM node and losing focus (state `q` survives because it lives in `Search`). *Fix:* hoist `Box` to module scope with props (`value`, `onChange`), or inline the `<input>`. *Verify:* type several characters and keep focus; add a mount/unmount log (`useEffect(() => { console.log("mount"); return () => console.log("unmount"); }, [])`) in `Box` and see only one mount; or watch "highlight updates" and confirm the input node isn't recreated (the Elements panel node stops flashing/replacing).

D2. *Cause:* stale closure plus missing cleanup. *Mechanism:* the effect with `[]` captures the first render's `seconds = 0`, so every tick sets `0 + 1 = 1` (stuck at 1). `setInterval` is never cleared, so the browser's timer keeps firing after unmount (a leak; in development with StrictMode the setup/cleanup/setup sequence creates **two** intervals, both setting 1, so you see jittery/duplicated work). *Fix:*

```
useEffect(() => {
  const id = setInterval(() => setSeconds(s => s + 1), 1000);
  return () => clearInterval(id);
}, []);
```

(Updater removes the dependency on `seconds`; cleanup clears the interval.) *Verify:* seconds count 1, 2, 3...; navigate away and confirm via a log in the interval callback (temporary) or the Performance monitor that no ticks continue; run under Strict Mode and confirm one tick per second (not two).

D3. *Defects:* (1) `titles.splice` mutates state and `setTitles(titles)` passes the same reference, so React bails out and doesn't re-render ("sometimes nothing changes"; it appears when something else re-renders). (2) `key={i}` index keys on rows whose `NoteRow` holds local state: after a removal, the state at index `i` stays at position `i`, so notes attach to the wrong titles. Additional invalid HTML: `<div>` as a direct child of `<ul>`; use `<li>`. *Fix:*

```
type Note = { id: string; title: string };
// Convert initial titles to items with stable ids once (e.g., in the parent's data source).
function remove(id: string) { setItems(prev => prev.filter(n => n.id !== id)); }
// <li key={n.id}> <NoteRow title={n.title} /> <button onClick={() => remove(n.id)}>Delete
{n.title}</button> </li>
```

Stable identity per item (not index) and an immutable update. *Verify:* type a note into row 2, delete row 1; the note stays with its title; in DevTools the `NoteRow` instances keep their `key`; also check that deleting updates immediately (the Profiler shows a commit).

D4. *Cause:* `useState(user.name)` only seeds at mount; the same `UserForm` instance (same type/position/key) persists across selection changes. *Mechanism:* new `user` props arrive but the state initialiser is ignored on updates, so the stale `name` stays. *Fix:* `<UserForm key={user.id} user={user} />`, which gives a new identity per user, resetting state in the first commit. (Alternatives: lift `name` state; controlled by parent; for non-editable display just derive from props.) *Verify:* select users A and B and confirm the input resets; DevTools shows a different `key` and a new instance (state resets); no effect-based reset flash.

D5. *Cause:* `onClick={setLikes(likes + 1)}` **calls** `setLikes` during render and passes its return value (`undefined`) as the handler. *Mechanism:* the call schedules an update during rendering, causing another render, which calls it again, an infinite loop; React stops it with "Too many re-renders". *Fix:* `onClick={() => setLikes(l => l + 1)}`. *Double-click question:* `() => setLikes(likes + 1)` computes from the snapshot `likes`; if both click events are processed before the re-render (rare in practice because React flushes discrete events like clicks before the next one, so each click usually sees the updated value), both would enqueue the same replacement. In normal browsers React flushes the render between two separate discrete click events, so both forms typically produce +2; the updater form is the one that is *guaranteed* correct whenever updates are queued together. *Verify:* click once and see +1, double-click and see +2; test in a unit test using `userEvent.dblClick`; no console error.

8.5 Part E answer key and evaluation criteria (5 points each)

Scoring (5): correct diagnosis/principle (2); at least two options with trade-offs (1); justified recommendation (1); a case where the recommendation fails (1).

E1. *Diagnosis:* the search text state lives in a component that also renders the chart (or an ancestor above both), so each keystroke re-renders the chart's subtree: React renders children by default when the parent renders. *Options:* (a) **colocate state**: move the search input and its state down into a `SearchPanel` that owns the text and the results, leaving the chart in a sibling that never sees the keystrokes (works when the chart doesn't need the text); (b) **composition**: pass the chart as `children`/a prop from a higher component that doesn't hold the state (the element is created by an owner that doesn't re-render, so React reuses the same element reference and skips re-rendering it); (c) `React.memo(Chart)` with stable props; (d) defer the result rendering with `useDeferredValue` / transitions (later phases). *Recommendation:* (a) is simplest and removes the cause, no memo bookkeeping. *Fails when:* the chart *does* depend on the search text (state must be shared above both), or the render cost isn't from the chart (profile first). Verify with the Profiler "why did this render".

E2. (a) 3 fields: either style; **uncontrolled with** `FormData` **on submit** (or controlled if live validation/formatting/disabling is wanted) is enough; simplest code, minimal renders; use native validation attributes plus server validation. (b) 40 fields with conditional sections and live validation: use a form library (React Hook Form–style, mostly uncontrolled registration; verify library docs in Phase 3/8) or a reducer-based controlled design with localised state to prevent whole-form re-renders on each keystroke; conditional sections depend on specific field values, so those values need to be observed (controlled subscriptions for just those fields). Trade-offs: controlled gives immediate access/consistency but can re-render a lot; uncontrolled reduces renders but makes cross-field logic and reset behaviour trickier. *Recommendation fails:* (a) when the signup needs live password-strength feedback; (b) when the form has 40 fields but trivial logic and acceptable performance, where plain controlled inputs are simpler to maintain.

E3. (1) **Reset on switch:** `<Composer key={conversationId} />` with local draft state; risk: unsaved text lost silently and a remount per switch (cost small). (2) **Remember per conversation:** lift drafts into a parent map `Record<conversationId, string>` (or a store) and make the composer controlled by `drafts[id]`; no key needed; risk: memory growth if never pruned; drafts lost on reload. (3) **Survive reload:** persist drafts to `localStorage`/`sessionStorage` or server-side (debounced writes), initialised lazily; risks: privacy on shared devices (scope to the user and clear on logout), quota/JSON errors

(try/catch), multi-tab conflicts (`storage` event), and sensitive content. Full credit needs the three mechanisms matched to the three requirements.

E4. *Feedback:* (1) `filteredItems` is **derived data**: compute it during render (`const filteredItems = items.filter(...)` , with `useMemo` only if profiling shows cost); the effect version renders once with stale data, then again after the effect sets state (flash, extra work), can drift out of sync, and adds code. (2) Analytics on submit belongs in the **submit handler**: an effect watching `submitted` fires on mount, runs twice under Strict Mode in development, and re-fires if the component remounts or the dependency toggles; events caused by a user action should be sent from that action. *Where an effect is right:* synchronizing with an external system, e.g., subscribing to a WebSocket for the current `roomId` , or setting up a non-React widget, or tracking a **page view** when the route (an external, navigation-driven change) changes: even then, ensure it tolerates the Strict Mode double run. Tone: explain the principle, propose the minimal diff, point to the lesson, and offer to pair.

---

9. Scoring sheet

Part	Category	Max	Your score
A (20 × 2)	Explanation	40	___
B (10 × 2)	Reasoning	20	___
C (4 × 10)	Implementation	40	___
D (5 × 8)	Debugging	40	___
E (4 × 5)	Reasoning	20	___
Explanation	A	40	___ (need ≥ 28)
Reasoning	B + E	40	___ (need ≥ 28)
Implementation	C	40	___ (need ≥ 28)
Debugging	D	40	___ (need ≥ 28)
Total		160	___ (need ≥ 128)

---

10. Scoring rules (apply uniformly)

- **Part A:** 2 = correct and complete in your own words; 1 = partly correct or missing a required element named in the key; 0 = incorrect or only restating the question. Do not award credit for memorised phrases without a mechanism.
- **Part B:** 1 point for the right observable result, 1 point for the right reason. A correct result with a wrong or missing reason earns 1. If your prediction was changed after running the code, score the original.
- **Part C:** use the rubrics. Code that does not compile loses the "compiles/lints" point *and* is capped at 7/10 overall; code that compiles but violates a core principle (mutation of state, effects for derived data, handler called in render) can score at most 6/10 even if it appears to work.
- **Part D:** the four sub-points are independent. "Fix" must actually remove the root cause; "silencing" (e.g., `// eslint-disable` , `any` , removing StrictMode) earns 0 for the fix.

- **Part E:** use the five-point criteria in §8.5. Answers that name only one option or give no trade-off are capped at 3.

---

11. Rubric interpretation and remediation map

Category pass criteria (each ≥ 28/40)

Category	What passing demonstrates	If below 28/40, revisit
Explanation	You can state the Phase 1 model in your own words with mechanisms, not slogans	Lessons 01–05 (model, declarative, elements), then 06, 12, 13, 17
Reasoning	You can predict behaviour from the model (snapshots, queues, identity, rendering) and defend design choices	13, 14, 15, 16, 09, 08
Implementation	You can build typed, accessible, correct components with state and effects	06–11, 15, 17, 19
Debugging	You can trace symptoms to root causes and verify fixes with tooling	16, 17, 18, 10, 09

Question-to-lesson map

Question	Lesson(s)
A1, A2, A3	01, 02, 03
A4, A5	04, 05
A6, A7	06, 07
A8, A9	08, 09
A10, A11	10, 11
A12, A13, A14, A20	12, 13, 14
A15	15
A16	16
A17	17
A18	18
A19	19
B1, B2, B10	14, 15
B3, B8, B9	08, 12, 11
B4, B5	09, 16
B6, B7	17
C1	06, 08, 09, 10
C2	10, 11, 12
C3	08, 09, 14, 15

Question	Lesson(s)
C4	14, 15, 17
D1, D4	16
D2	14, 15, 17
D3	09, 12, 16
D5	10, 14, 15
E1	07, 13, 18
E2	11
E3	16
E4	12, 17

12. Promotion criteria

You may proceed to Phase 2 — Rendering, reconciliation, DOM, internals when **all** of the following hold:

1. **Overall score ≥ 128 / 160 (80%).**
2. **Each category ≥ 28 / 40 (70%):** Explanation, Reasoning, Implementation, Debugging. Passing the conceptual part alone does not advance you; a high Explanation score cannot compensate for a failing Implementation or Debugging score.
3. **Part C compiled and linted cleanly** in your own project (no `any`, no suppressed lint rules).
4. **You verified at least D1, D2 and D4 in the browser with React DevTools** (observe the remount, the leaked interval or stale closure, and the preserved state) rather than relying on the answer key alone.
5. **Project 1 — Interactive fundamentals app** (see the `projects/` directory) is completed to the point where its review checklist passes. This is the practical phase gate listed in the roadmap; the assessment tests knowledge, the project tests sustained application.

If you do not pass: use the remediation map to revisit only the lessons for the failed categories; redo the *relevant* parts after 48 hours; explain your corrected answers aloud (or in writing) to someone else or to yourself without notes. Repeat until the thresholds hold. Advancing with a weak Phase 1 foundation (state snapshots, keys/identity, purity) makes Phase 2's internals material (reconciliation, Fiber, scheduling) much harder, because it explains *why* those behaviours exist rather than re-teaching what they are.

If you pass with a category within 5 points of the threshold: proceed, but re-read the relevant lesson's Section 12 answers while studying Phase 2; the same ideas return with mechanisms attached.